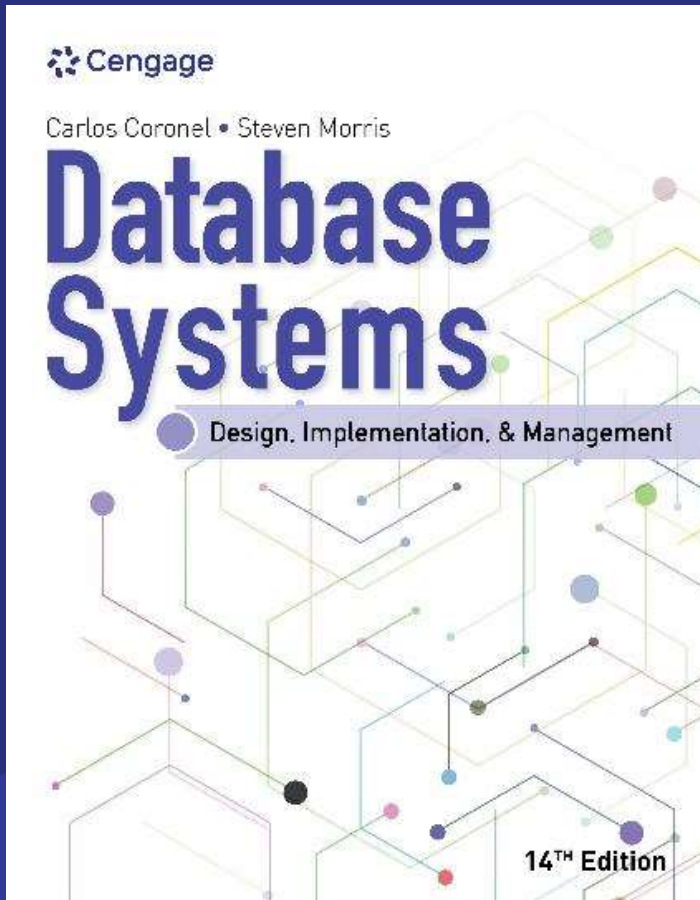




Database Systems: Design, Implementation, and Management, 14e

Module 8: Advanced SQL

Chapter Objectives

By the end of this chapter, you should be able to:

1. Use SQL to create a table manually
2. Use SQL to create a copy of a table using a subquery
3. Manipulate the structure of tables to add, modify, and remove columns and constraints
4. Use SQL to do data manipulation (insert, update, and delete rows of data)
5. Use SQL to create database views, including updatable views
6. Use procedural SQL to create triggers, stored procedures, and procedural SQL functions

Data Definition Commands (1 of 6)

- Starting database model
 - Refer to Figure 8.1 on the following slide
- Creating the Database
 - Before a new RDBMS can be used, the database structure and the tables that will hold the end-user data must be created
- The Database Schema
 - A **schema** is a logical group of database objects—such as tables and indexes—that are related to each other

Data Definition Commands (2 of 6)

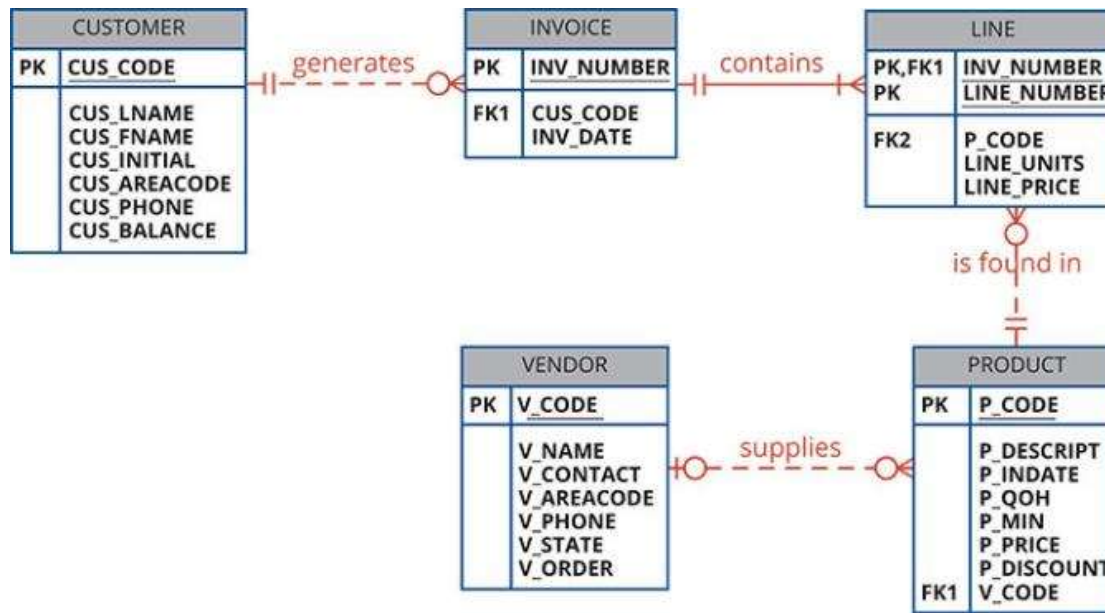


Figure 8.1 Database Model

Data Definition Commands (3 of 6)

Table name: VENDOR

Database name: Ch08_SaleCo

V_CODE	V_NAME	V_CONTACT	V_AREACODE	V_PHONE	V_STATE	V_ORDER
21225	Bryson, Inc.	Smithson	615	223-3234	TN	Y
21226	SuperLoo, Inc.	Flushing	904	215-8995	FL	N
21231	D&E Supply	Singh	615	228-3245	TN	Y
21344	Gomez Bros.	Ortega	615	889-2546	KY	N
22567	Dome Supply	Smith	901	678-1419	GA	N
23119	Randssets Ltd.	Anderson	901	678-3998	GA	Y
24004	Brackman Bros.	Browning	615	228-1410	TN	N
24288	ORDVA, Inc.	Hakford	615	898-1234	TN	Y
25443	B&K, Inc.	Smith	904	227-0093	FL	N
25501	Damal Supplies	Smythe	615	890-3529	TN	N
25595	Rubicon Systems	Orton	904	456-0092	FL	Y

Table name: PRODUCT

P_CODE	P_DESCRIPT	P_INDATE	P_QOH	P_MIN	P_PRICE	P_DISCOUNT	V_CODE	P_MIN_ORDER	P_REORDER
11QER/31	Power painter, 15 psi., 3-nozzle	03-Nov-21	8	5	109.99	0.00	25595	25	0
13-Q2/P2	7.25-in. pwr. saw blade	13-Dec-21	32	15	14.99	0.05	21344	50	0
14-Q1/L3	9.00-in. pwr. saw blade	13-Nov-21	18	12	17.49	0.00	21344	50	0
1546-QQ2	Hrd. cloth, 1/4-in., 2x50	15-Jan-22	15	8	39.95	0.00	23119	35	0
1558-QW1	Hrd. cloth, 1/2-in., 3x50	15-Jan-22	23	5	43.99	0.00	23119	25	0
2232/QTY	B&D jigsaw, 12-in. blade	30-Dec-21	8	5	109.92	0.05	24288	15	0
2232/QWE	B&D jigsaw, 8-in. blade	24-Dec-21	6	5	99.87	0.05	24288	15	0
2238/QPD	B&D cordless drill, 1/2-in.	20-Jan-22	12	5	38.95	0.05	25595	12	0
23109-HB	Claw hammer	20-Jan-22	23	10	9.95	0.10	21225	25	0
23114-AA	Sledge hammer, 12 lb.	02-Jan-22	8	5	14.4	0.05		12	0
54778-2T	Rat-tail file, 1/8-in. fine	15-Dec-21	43	20	4.99	0.00	21344	25	0
89-WRE-Q	Micut chain saw, 16 in.	07-Feb-22	11	5	256.99	0.05	24288	10	0
PVC23ORT	PVC pipe, 3.5-in., 8-ft	20-Feb-22	188	75	5.87	0.00		50	0
SM-18277	1.25-in. metal screw, 25	01-Mar-22	172	75	6.99	0.00	21225	50	0
SW-23116	2.5-in. wd. screw, 50	24-Feb-22	237	100	8.45	0.00	21231	100	0
WR3/TT3	Steel matting, 4'x8'x1/6", .5" mesh	17-Jan-22	18	5	119.95	0.10	25595	10	0

Figure 8.2 Vendor and Product Tables

Data Definition Commands (4 of 4)

Table 8.1 Some Common SQL Data Types		
Data Type	Format	Comments
Numeric	NUMERIC(L,D) or NUMBER (L,D)	The declaration NUMERIC(7,2) or NUMBER (7,2) indicates that numbers will be stored with two decimal places and may be up to seven digits long, including the sign and the decimal place (for example, 12.32 or -134.99).
	INTEGER	May be abbreviated as INT. Integers are (whole) counting numbers, so they cannot be used if you want to store numbers that require decimal places.
	SMALLINT	Like INTEGER but limited to integer values up to six digits. If your integer values are relatively small, use SMALLINT instead of INT.
	DECIMAL(L,D)	Like the NUMERIC specification, but the storage length is a <i>minimum</i> specification. That is, greater lengths are acceptable, but smaller ones are not. DECIMAL(9,2), DECIMAL(9), and DECIMAL are all acceptable. In most popular DBMSs, this difference is not implemented so NUMERIC and DECIMAL operate the same.
Character	CHAR(L)	Fixed-length character data for up to 255 characters. If you store strings that are not as long as the CHAR parameter value, the remaining spaces are left unused. Therefore, if you specify CHAR(25), strings such as <i>Smith</i> and <i>Katzenjammer</i> are each stored as 25 characters. However, a U.S. area code is always three digits long, so CHAR(3) would be appropriate if you wanted to store such codes.
	VARCHAR(L) or VARCHAR2(L)	Variable-length character data. The designation VARCHAR(25) or VARCHAR2(25) will let you store up to 25 characters. However, unlike CHAR, VARCHAR will not leave unused spaces. Oracle automatically converts VARCHAR to VARCHAR2.
Date	DATE	Stores dates in the Julian date format.

Creating Table Structures (1 of 3)

- CREATE TABLE command
 - **CREATE TABLE** is an SQL command that creates a table's structures using the characteristics and attributes given
 - The following is the syntax to use in order to implement a table structure

```
CREATE TABLE tablename (  
    column1          data type          [constraint] [,  
    column2          data type          [constraint] ] [,  
    PRIMARY KEY      (column1          [, column2]) ] [,  
    FOREIGN KEY      (column1          [, column2]) REFERENCES tablename] [,  
    CONSTRAINT       constraint ] );
```

Creating Table Structures (2 of 3)

- SQL Constraints
 - The FOREIGN KEY constraint ensures that you cannot delete a vendor from the VENDOR table if at least one product row references that vendor
 - The NOT NULL constraint ensures that a column does not accept nulls
 - The UNIQUE constraint ensures that all values in a column are unique
 - The DEFAULT constraint assigns a value to an attribute when a new row is added to a table
 - The CHECK constraint is used to validate data when an attribute value is entered

Creating Table Structures (3 of 3)

- Create a Table with a SELECT Statement
 - SQL provides a way to rapidly create a new table based on selected columns and rows of an existing table using a subquery
 - All of the data rows returned by the SELECT statement are copied automatically
- SQL Indexes
 - **CREATE INDEX** improves the efficiency of searches and avoids duplicate column values
 - **DROP INDEX** is an SQL command used to delete database objects such as tables, views, indexes, and users

Knowledge Check Activity 8-1

- **What is the purpose of a CHECK constraint?**

Knowledge Check Activity 8-1: Answer

- **What is the purpose of a CHECK constraint?**
- **Answer:** A CHECK constraint is used to limit the values that can appear in an attribute. It performs the function of enforcing a domain.

Altering Table Structures (1 of 4)

- All changes in the table structure are made by using the **ALTER TABLE** command followed by a keyword that produces the specific change you want to make
 - The following three options are available: ADD, MODIFY, and DROP
- Changing a Column's Data Type
 - ALTER can be used to change a data type
- Changing a Column's Data Characteristics
 - If the column to be changed already contains data, you can make changes in the column's characteristics if those changes do not alter the data type

Altering Table Structures (2 of 4)

- Adding a Column
 - You can alter an existing table by adding one or more columns
 - Be careful not to include the NOT NULL clause for the new column

Altering Table Structures (3 of 4)

- Adding Primary Key, Foreign Key, and Check Constraints

- Primary key syntax is as follows:

```
ALTER TABLE PART
ADD PRIMARY KEY (PART_CODE);
```

- Foreign key syntax is as follows:

```
ALTER TABLE PART
ADD FOREIGN KEY (V_CODE) REFERENCES
VENDOR;
```

- Check constraint syntax is as follows:

```
ALTER TABLE PART
ADD CHECK (PART_PRICE >= 0);
```

Altering Table Structures (4 of 4)

- Dropping a Column

- The following is the syntax:

```
ALTER TABLE VENDOR  
DROP COLUMN V_ORDER;
```

- Deleting a Table From the Database

- **DROP TABLE** is an SQL command used to remove database tables
- The following is the syntax:

```
DROP TABLE PART;
```

Data Manipulation Commands (1 of 4)

- Adding Table Rows
 - **INSERT** is a SQL command that allows the insertion of one or more data rows into a table using a subquery
 - INSERT command syntax:

```
INSERT INTO tablename VALUES (value1, value2, ..., valuen)
```
 - To inserting rows with null attributes, use a NULL entry
 - When inserting rows with optional attributes, you can indicate attributes that have required values

Data Manipulation Commands (2 of 4)

- Inserting Table Rows with a SELECT Subquery
 - Using a subquery with the INSERT command, it is possible to add multiple rows to a table, using another table as the source, at the same time
 - Use the following SELECT syntax:

```
INSERT INTO    target_tablename[(target_columnlist)]  
SELECT        source_columnlist  
FROM          source_tablename;
```

- Saving Table Changes
 - The syntax for the **COMMIT** command is:

```
COMMIT [WORK | TRANSACTION]
```

Data Manipulation Commands (3 of 4)

- Updating Table Rows
 - **UPDATE** command is used to modify data in a table
 - The syntax for the UPDATE command is as follows:

```
UPDATE      tablename
SET         columnname = expression [, columnname =
```

expression]

```
[WHERE      conditionlist ];
```

Data Manipulation Commands (4 of 4)

- Deleting Table Rows

- The following is the **DELETE** statement syntax:

```
DELETE FROM tablename  
[WHERE conditionlist ];
```

- Restoring Table Contents

- The ROLLBACK command is used to restore the database table contents to the condition that existed after the last COMMIT statements

```
ROLLBACK;
```

Virtual Tables: Creating a View (1 of 2)

- A **view** is a virtual table based on a SELECT query that is saved as an object in the database
- A **base table** is the table on which a view is based
- **CREATE VIEW** is a SQL command that creates a logical, “virtual” table
 - The view can be treated as a real table
 - The following is the CREATE VIEW syntax:

`CREATE VIEW viewname AS SELECT query`

Virtual Tables: Creating a View (2 of 2)

```
1 CREATE VIEW PRICEGT50 AS
2     SELECT P_DESCRIPTOR, P_QOH, P_PRICE
3     FROM PRODUCT
4     WHERE P_PRICE > 50;
5
6 • SELECT * FROM PRICEGT50;
7
```



Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

P_DESCRIPTOR	P_QOH	P_PRICE
Power painter, 15 psi., 3-nozzle	8	109.99
B&D jigsaw, 12-in. blade	8	109.92
B&D jigsaw, 8-in. blade	6	99.87
Hicut chain saw, 16 in.	11	256.99
Steel matting, 4'x8'x1/6", .5" mesh	18	119.95

Figure 8.5 Creating a Virtual Table with the Create View command

Updatable Views

- One of the most common operations in production database environments is to use batch update routines to update a master table attribute with transaction data
- A **batch update routine** is a routine that pools multiple transactions into a single batch to update a master table in a single operation
- A view that can update attributes in base tables that are used in the view is known as an **updatable view**
- The following are the most common updatable view restrictions:
 - GROUP BY expressions or aggregate functions cannot be used
 - Set operators cannot be used
 - Most restrictions are based on the use of JOINS or group operators in views

Knowledge Check Activity 8-2

- **What is the difference between an INSERT command and an UPDATE command?**

Knowledge Check Activity 8-2: Answer

- **What is the difference between an INSERT command and an UPDATE command?**
- **Answer:** The INSERT command is used to add a new row to a table. The UPDATE command changes the values in attributes of an existing row. UPDATE will not increase the number of rows in a table, but INSERT will.

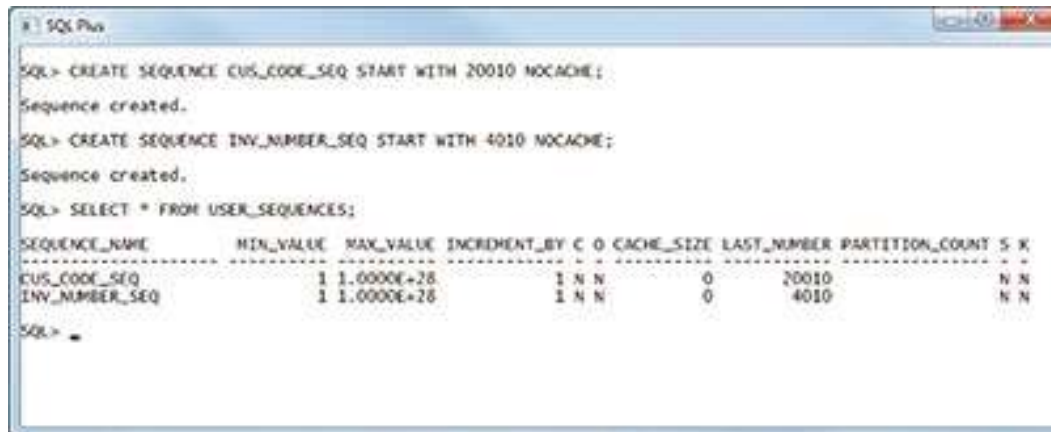
Auto Increment, Identity, and Sequences (1 of 3)

- MySQL uses the AUTO_INCREMENT property during table creation to indicate that values for an attribute should be generated in the same fashion
 - Similar to IDENTITY columns, only one column in a table can have AUTO_INCREMENT specified, and that column must also be defined as the primary key of the table
- A sequence is an object for generating unique segment values for a field
- In recent years, Oracle added support for Identity columns and MS SQL Server added support for sequences

Auto Increment, Identity, and Sequences (2 of 3)

- The following are similarities in the use of sequences across multiple DBMSs
 - Sequences are an independent object in the database
 - Sequences have a name and can be used anywhere a value expected
 - Sequences are not tied to a table or a column
 - Sequences generate a numeric value that can be assigned to any column in any table
 - The table attribute with an assigned value can be edited and modified

Auto Increment, Identity, and Sequences (3 of 3)



```
SQL> CREATE SEQUENCE CUS_CODE_SEQ START WITH 20010 NOCACHE;  
Sequence created.  
SQL> CREATE SEQUENCE INV_NUMBER_SEQ START WITH 4010 NOCACHE;  
Sequence created.  
SQL> SELECT * FROM USER_SEQUENCES;
```

SEQUENCE_NAME	MIN_VALUE	MAX_VALUE	INCREMENT_BY	C	O	CACHE_SIZE	LAST_NUMBER	PARTITION_COUNT	S	K
CUS_CODE_SEQ	1	1.0000E+28	1	N	N	0	20010		N	N
INV_NUMBER_SEQ	1	1.0000E+28	1	N	N	0	4010		N	N

```
SQL>
```

Figure 8.7 Oracle Sequence

Procedural SQL

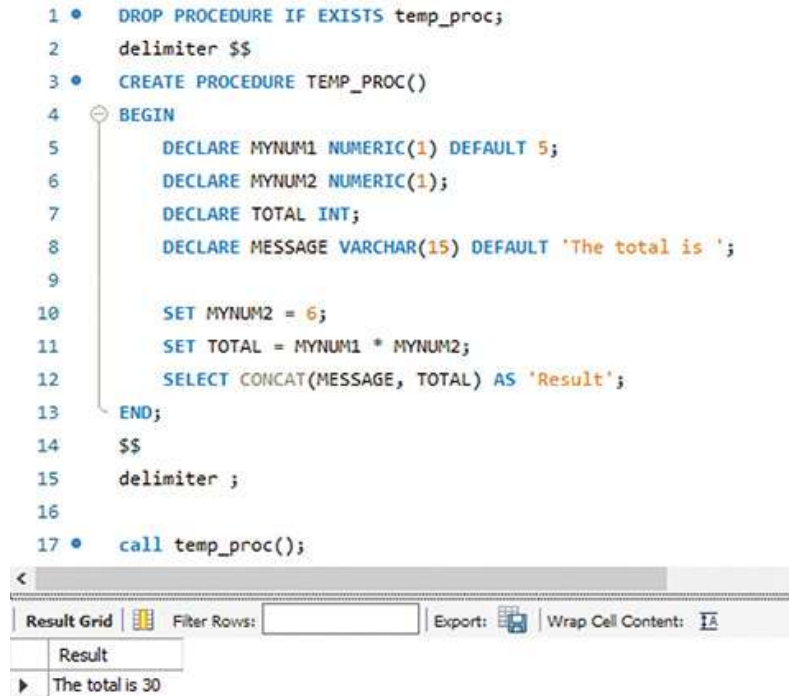
- Persistent stored module (PSM) is a block of code that contains standard SQL statements and procedural extensions that is stored and executed at the DBMS server
- Procedural SQL is an extension of SQL that adds procedural programming capabilities, such as variables and logical flow control, to SQL and is designed to run inside the database
 - The procedural code is executed as a unit by the DBMS when it is invoked by the end user
- End users can use procedural SQL to create the following:
 - Stored procedures
 - Triggers

Stored Procedures

- A **stored procedure** is a named collection of procedural and SQL statements
 - They are stored in the database
 - A major advantage of stored procedures is that they can be used to encapsulate and represent business transactions
- Using stored procedures offers the following advantages:
 - Stored procedures substantially reduce network traffic and increase performance
 - Stored procedures help reduce code duplication by means of code isolation and code sharing

Working with Variables

```
1 • DROP PROCEDURE IF EXISTS temp_proc;
2   delimiter $$
3 • CREATE PROCEDURE TEMP_PROC()
4   BEGIN
5       DECLARE MYNUM1 NUMERIC(1) DEFAULT 5;
6       DECLARE MYNUM2 NUMERIC(1);
7       DECLARE TOTAL INT;
8       DECLARE MESSAGE VARCHAR(15) DEFAULT 'The total is ';
9
10      SET MYNUM2 = 6;
11      SET TOTAL = MYNUM1 * MYNUM2;
12      SELECT CONCAT(MESSAGE, TOTAL) AS 'Result';
13  END;
14  $$
15  delimiter ;
16
17 • call temp_proc();
```



Result
The total is 30

Figure 8.11 Variable Manipulation

Conditional Execution

```
1 * DROP PROCEDURE IF EXISTS temp_proc;
2   delimiter $$
3 * CREATE PROCEDURE TEMP_PROC()
4 BEGIN
5     DECLARE MYNUM1 INT DEFAULT 5;
6     DECLARE MYNUM2 INT DEFAULT 60;
7     DECLARE TOTAL INT;
8     DECLARE MESSAGE VARCHAR(50);
9
10    SET TOTAL = MYNUM1 * MYNUM2;
11    IF TOTAL < 100 THEN
12        SET MESSAGE = 'The total is small: ';
13    ELSE
14        SET MESSAGE = 'The total is large: ';
15    END IF;
16    SELECT CONCAT(MESSAGE, TOTAL) AS 'Result';
17 END;
18 $$
19 delimiter ;
20
21 * call temp_proc();
```



Result
The total is large: 300

Figure 8.12 Conditional Execution

Iteration or Looping

```
1 * DROP PROCEDURE IF EXISTS temp_proc;
2 delimiter $$
3 * CREATE PROCEDURE TEMP_PROC()
4 BEGIN
5     DECLARE MYNUM INT DEFAULT 1;
6     DECLARE RESULT VARCHAR(100) DEFAULT MYNUM;
7     COUNTING LOOP
8     SET MYNUM = MYNUM + 1;
9     IF MYNUM > 20 THEN
10        LEAVE COUNTING;
11    END IF;
12    SET RESULT = CONCAT(RESULT, ', ', MYNUM);
13    END LOOP;
14    SELECT RESULT;
15 END;
16 $$
17 delimiter ;
18 * call temp_proc();
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: 15

RESULT

1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20

• **Figure 8.14** A Simple Loop

```
1 * DROP PROCEDURE IF EXISTS temp_proc;
2 delimiter $$
3 * CREATE PROCEDURE TEMP_PROC()
4 BEGIN
5     DECLARE MYNUM INT DEFAULT 1;
6     DECLARE RESULT VARCHAR(100) DEFAULT MYNUM;
7     WHILE MYNUM < 20 DO
8         SET MYNUM = MYNUM + 1;
9         SET RESULT = CONCAT(RESULT, ', ', MYNUM);
10    END WHILE;
11    SELECT RESULT;
12 END;
13 $$
14 delimiter ;
15 * call temp_proc();
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: 15

RESULT

1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20

• **Figure 8.15** Implementation of a WHILE Loop

Select Processing with Cursors (1 of 2)

- A **cursor** is a special construct used to hold data rows returned by a SQL query
 - An **implicit cursor** is automatically created when the SQL statement returns only one row
 - An **explicit cursor** holds the output of a SQL statement that may return two or more rows but could return zero or only one row
 - To create an explicit cursor, you use the following syntax:

`DECLARE cursor_name CURSOR FOR select-query;`

- Cursor-style processing involves retrieving data from the cursor one row at a time
 - When you fetch a row from the cursor, the data from the “current” row in the cursor is copied to the PL/SQL variables

Select Processing with Cursors (2 of 2)

```

3 DROP PROCEDURE IF EXISTS temp_proc;
4 delimiter $$
5 CREATE PROCEDURE TEMP_PROC()
6 BEGIN
7     DECLARE W_P_CODE VARCHAR(15);
8     DECLARE W_P_DESCRIPTOR VARCHAR(50);
9     DECLARE W_TOT INT DEFAULT 0;
10    DECLARE NOTFOUND INT DEFAULT 0;
11    DECLARE PROD_CURSOR CURSOR FOR SELECT P_CODE, P_DESCRIPTOR
12        FROM PRODUCT
13        WHERE P_QOH > (SELECT AVG(P_QOH) FROM PRODUCT);
14    DROP TEMPORARY TABLE IF EXISTS MYTAB;
15    CREATE TEMPORARY TABLE MYTAB (RESULT VARCHAR(60));
16    INSERT INTO MYTAB VALUES ('*****');
17    OPEN PROD_CURSOR;
18    GET_PRODUCTS: LOOP
19        FETCH PROD_CURSOR INTO W_P_CODE, W_P_DESCRIPTOR;
20        IF NOTFOUND = 1 THEN
21            LEAVE GET_PRODUCTS;
22        END IF;
23        INSERT INTO MYTAB VALUES (CONCAT(W_P_CODE, ' ' || W_P_DESCRIPTOR));
24        SET W_TOT = W_TOT + 1;
25    END LOOP;
26    CLOSE PROD_CURSOR;
27    INSERT INTO MYTAB VALUES ('*****');
28    INSERT INTO MYTAB VALUES (CONCAT('TOTAL PRODUCTS PROCESSED:', W_TOT));
29    INSERT INTO MYTAB VALUES ('--- END OF REPORT ---');
30    SELECT RESULT AS 'PRODUCTS WITH P_QOH > AVG(P_QOH)' FROM MYTAB;
31    DROP TEMPORARY TABLE MYTAB;
32END;
33$$
34 delimiter ;
35 call temp_proc();

```

The screenshot shows a SQL IDE with two panes. The top pane contains the SQL code for a stored procedure named TEMP_PROC. The code drops the procedure if it exists, creates it with a loop to fetch products where P_QOH is greater than the average P_QOH, and inserts the results into a temporary table MYTAB. The bottom pane shows the results of the procedure execution, which lists products with P_QOH greater than the average P_QOH, followed by the total count of products processed.

PRODUCTS WITH P_QOH > AVG(P_QOH)

PVC300F -> PVC pipe, 3.5-in, 0 ft
SH-1827T -> 1.25-in. metal screw, 25
SW-22136 -> 2.5-in. wd. screw, 50

TOTAL PRODUCTS PROCESSED: 4
--- END OF REPORT ---

Figure 8.19 An Explicit Cursor Example

Stored Procedures with Parameters (1 of 2)

- One of the most valuable features of working with stored procedures is their ability to use parameters
- A parameter is a value that is provided to the program at the time of execution
- Figure 8.21 shows the code necessary to pass an argument, or parameter, to represent a rate of increase to the procedure

Stored Procedures with Parameters (2 of 2)

```
1 * DROP PROCEDURE IF EXISTS prc_prod_discount;
2 delimiter $$
3 * CREATE PROCEDURE PRC_PROD_DISCOUNT(IN MPI DECIMAL(2,2))
4 BEGIN
5     IF MPI <= 0 OR MPI >= 1 THEN -- validate MPI parameter
6         SELECT 'Error: Value must be greater than 0 and less than 1' AS RESULT;
7     ELSE
8         UPDATE PRODUCT
9         SET P_DISCOUNT = P_DISCOUNT + MPI
10        WHERE P_QOH >= P_MIN * 2;
11        SELECT '*** Update finished.';
12    END IF;
13 END;
14 $$
15 delimiter ;
16
17 * START TRANSACTION;
18 * call PRC_PROD_DISCOUNT(.33);
19 * SELECT P_CODE, P_DESCRIPT, P_QOH, P_MIN, P_DISCOUNT FROM PRODUCT;
20 * ROLLBACK;
```

P_CODE	P_DESCRIPT	P_QOH	P_MIN	P_DISCOUNT
13QER/31	Power painter, 15 psi., 3-nozzle	8	5	0.00
13-Q2/P2	7.25-in. pwr. saw blade	32	15	0.38
14-Q1/L3	9.00-in. pwr. saw blade	18	12	0.00
1596-QQ2	Hrd. cloth, 1/4-in., 2x50	15	8	0.00
1558-Q/U1	Hrd. cloth, 1/2-in., 3x50	23	9	0.33
2232/QTY	8&D jgsaw, 12-in. blade	8	5	0.05
2232/QVE	8&D jgsaw, 8-in. blade	6	5	0.05
2238/QFD	8&D cordless drill, 1/2-in.	12	5	0.38
23109-HB	Claw hammer	23	10	0.43
23114-AA	Sledge hammer, 12 lb.	8	5	0.05
54778-ZT	Rat-tail file, 1/8-in. fine	43	20	0.33
89-WRE-Q	Hicut chain saw, 16 in.	11	5	0.38
PVC23DR7	PVC pipe, 3.5-in., 8-ft	188	75	0.33
SM-18277	1.25-in. metal screw, 25	172	75	0.33
SW-23116	2.5-in. wd. screw, 50	237	100	0.33
WR3/TT3	Steel matting, 4'x8'x1/6", .5" m...	18	5	0.43

Figure 8.21 Second Version of the PRC_PROD_DISCOUNT Stored Procedure

Triggers (1 of 2)

- A **trigger** is a procedural SQL code automatically invoked by the relational database management system when a data manipulation event occurs
- It is useful to remember the following:
 - A trigger is invoked before or after a data row is inserted, updated, or deleted
 - A trigger is associated with a database table
 - Each database table may have one or more triggers
 - A trigger is executed as part of the transaction that triggered it
 - Triggers are critical to proper database operation and management

Triggers (2 of 2)

- A trigger definition contains the following parts:
 - The triggering timing: indicates when the trigger's code executes
 - The triggering statement or triggering event: the statement that causes the trigger to execute
 - The triggering level:
 - A **statement-level trigger** is executed once and a **row-level trigger** requires use of the FOR EACH ROW keywords
 - The triggering action: the SQL and procedural SQL code enclosed between the BEGIN and END keywords

User Defined Functions

- A user defined function (UDF) is basically a named group of procedural and SQL statements that returns a value, as indicated by a RETURN statement in its program code
- To create a function, you use the following syntax:

```
CREATE FUNCTION function_name (IN argument data-type, ...) RETURNS data-type
```

```
BEGIN
```

```
    Procedure SQL statements;
```

```
    ...
```

```
    RETURN (value or expression);
```

```
END;
```

Embedded SQL (1 of 4)

- **Embedded SQL** are SQL statements contained within an application programming language
- A **host language** is any language that contains embedded SQL statements
- Differences between SQL and procedural languages include the following
 - Run-time mismatch:
 - With SQL, each instruction is parsed and it is executed one instruction at a time
 - The Host program typically runs at the client side in its own memory space

Embedded SQL (2 of 4)

- Differences between SQL and procedural languages (continued)
 - Processing mismatch
 - Conventional programming languages process one data element at a time
 - Newer programming environments manipulate data sets in a cohesive manner
 - Data type mismatch
 - Data types provided by SQL might not match data types used in different host languages

Embedded SQL (3 of 4)

- The embedded SQL framework defines the following:
 - A standard syntax to identify embedded SQL code within the host language
 - A standard syntax to identify host variables
 - A communication area used to exchange status and error information between SQL and the host language

Embedded SQL (4 of 4)

- **Static SQL** is a style of embedded SQL in which the SQL statements do not change while the application is running
- **Dynamic SQL** is a term used to describe an environment in which the SQL statement is generated at run time
 - In a dynamic SQL environment, a program can generate the SQL statements that are required to respond to ad hoc queries

Knowledge Check Activity 8-3

- **What is a stored procedure, and why is it particularly useful?**

Knowledge Check Activity 8-3: Answer

- **What is a stored procedure, and why is it particularly useful?**
- **Answer:** A stored procedure is a named block of Procedural SQL and standard SQL statements. One of the major advantages of stored procedures is that they can be used to encapsulate and represent business transactions.

Summary

Now that the lesson has ended, you should be able to:

1. Use SQL to create a table manually
2. Use SQL to create a copy of a table using a subquery
3. Manipulate the structure of tables to add, modify, and remove columns and constraints
4. Use SQL to do data manipulation (insert, update, and delete rows of data)
5. Use SQL to create database views, including updatable views
6. Use procedural SQL to create triggers, stored procedures, and procedural SQL functions